

LECTURE 5

COMPUTER ARITHMETIC

Lecture overview

- This lecture will cover the followings:
 - Addition
 - Subtraction
 - Concept of an overflow
 - Multiplication
 - Division

Addition and Subtraction

- Normal binary addition
- Monitor sign bit for overflow
- Take twos compliment of subtrahend and add to minuend
i.e. $a - b = a + (-b)$
- So we only need addition and complement circuits

Fixed Point Addition and Subtraction

TWO'S COMPLEMENT ADDITION AND SUBTRACTION

$$a - b = a + (-b)$$

Example of 8-bit two's complement addition, using two positive numbers:

$$\begin{array}{r} 00001010 (+10)_{10} \\ + 00010111 (+23)_{10} \\ \hline 00100001 (+33)_{10} \end{array}$$

Addition and Subtraction cont...

- A positive and a negative number can be added in a similar manner:

$$\begin{array}{r} 0\ 0\ 0\ 0\ 0\ 1\ 0\ 1\ (+5)_{10} \\ +\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 0\ (-2)_{10} \\ \hline \end{array}$$

Discard carry (1) 0 0 0 0 0 0 1 1 (+3)₁₀

- The carry produced by addition at the highest (leftmost) bit position is discarded in two's complement addition.

Addition and Subtraction cont...

$$\begin{array}{r} 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ (-1)_{10} \\ +\ 1\ 1\ 1\ 1\ 1\ 1\ 0\ 0\ (-4)_{10} \\ \hline \end{array}$$

Discard carry (1) $1\ 1\ 1\ 1\ 1\ 0\ 1\ 1\ (-5)_{10}$

- The carry out of the leftmost bit is discarded because the number system is **modular**— it “wraps around” from the largest positive number to the largest negative number

Addition and Subtraction cont...

Overflow

- When two numbers are added that have large magnitudes and the same sign, an **overflow** will occur if the result is too large to fit in the number of bits used in the representation.

Example of an Overflow

- Add $80_{10} + 80_{10}$

Using binary: $80 = 0101\ 0000$

$$\underline{80 = 0101\ 0000}$$

$$1010\ 0000_2 = -96_{10}$$

How: MSB = 1 hence a negative number

- Since we know that the largest positive 8-bit two's complement number is $+(127)_{10}$, and it is therefore impossible to represent $(+160)_{10}$.

Example of an Overflow

- In general, if two numbers of opposite signs are added, then an overflow cannot occur.
- This is because the magnitude of the result can be no larger than the magnitude of the larger operand.
- This leads to the definition of two's complement overflow:
 - *If the numbers being added are of the same sign and the result is of the opposite sign, then an overflow occurs and the result is incorrect.*

Example of an Overflow

- *If the numbers being added are of opposite signs, then an overflow will never occur.*
- *As an alternative method of detecting overflow for addition, an overflow occurs if and only if the carry into the sign bit differs from the carry out of the sign bit.*

Example of an Overflow

- *If a positive number is subtracted from a negative number and the result is positive, or if a negative number is subtracted from a positive number and the result is negative, then an overflow occurs.*
- *If the numbers being subtracted are of the same sign, then an overflow will never occur.*

MULTIPLICATION

- Compared with addition and subtraction, multiplication is a complex operation, whether performed in hardware or software.
- Multiplication of unsigned binary integers is handled similar to the way it is carried out by hand for decimal numbers.
- Each bit of the multiplier determines whether or not the multiplicand, shifted left according to the position of the multiplier bit, is added into the product.
- When two unsigned n -bit numbers are multiplied, the result can be as large as $2n$ bits.

Multiplication of two unsigned binary integers using Array Multiplier

Perform $11_{10} \times 13_{10}$ using array multiplier.

1011	Multiplicand (11_{10})
<u>x 1101</u>	Multiplier (13_{10})
1011	Partial product
0000	Partial product
1011	Partial product
<u>1011</u>	Partial product
10001111	Product (143_{10})

- Note: need double length result

MULTIPLICATION cont...

- Several important observations can be made from the example above:
 1. Multiplication involves the generation of partial products, one for each digit in the multiplier. These partial products are then summed to produce the final product.
 2. The partial products are easily defined. When the multiplier bit is 0, the partial product is 0. When the multiplier is 1, the partial product is the multiplicand.

MULTIPLICATION cont...

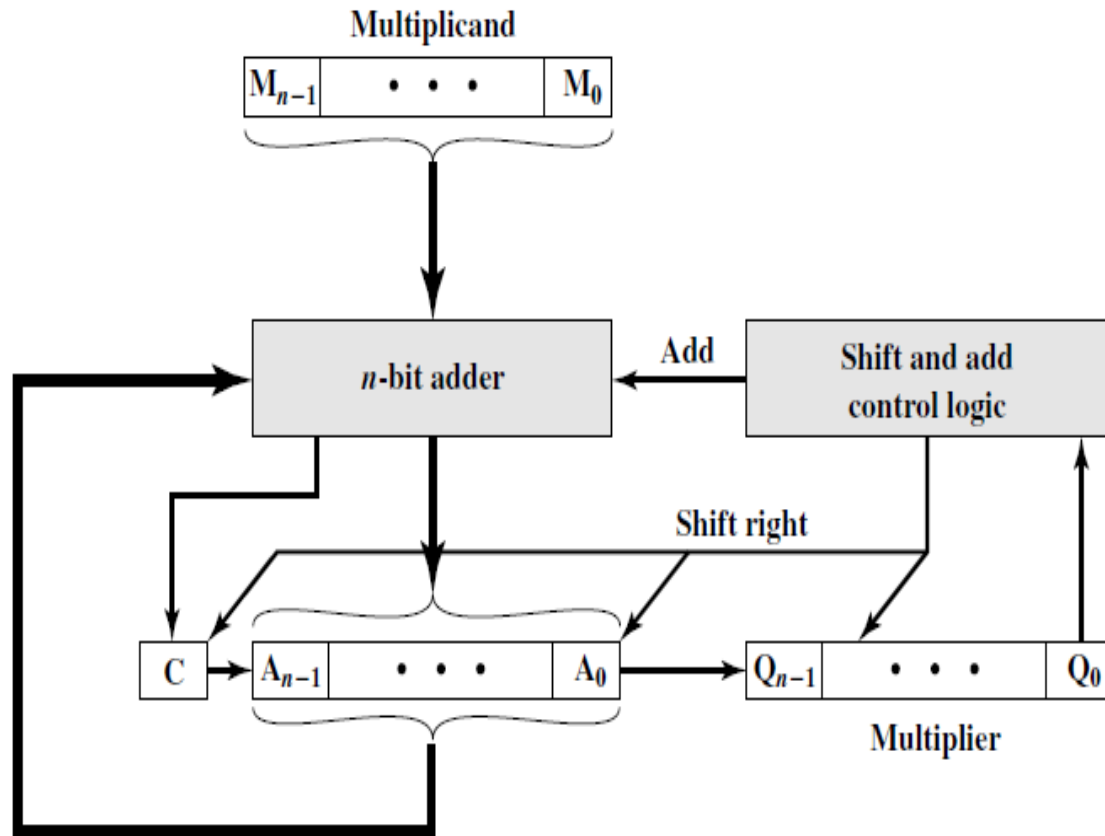
3.The total product is produced by summing the partial products. For this operation, each successive partial product is shifted one position to the left relative to the preceding partial product.

4.The multiplication of two n -bit binary integers results in a product of up to **$2n$** bits in length

MULTIPLICATION cont...

- Compared with the pencil-and-paper approach, there are things which can be done to make computerized multiplication more efficient.
 - ❑ First, perform a running addition on the partial products rather than waiting until the end.
 - This eliminates the need for storage of all the partial products; fewer registers are needed.
 - ❑ Second, time can be saved on the generation of partial products.
 - For each 1 on the multiplier, an add and a shift operation are required; but for each 0, only a shift is required.

Block diagram of a multiplier



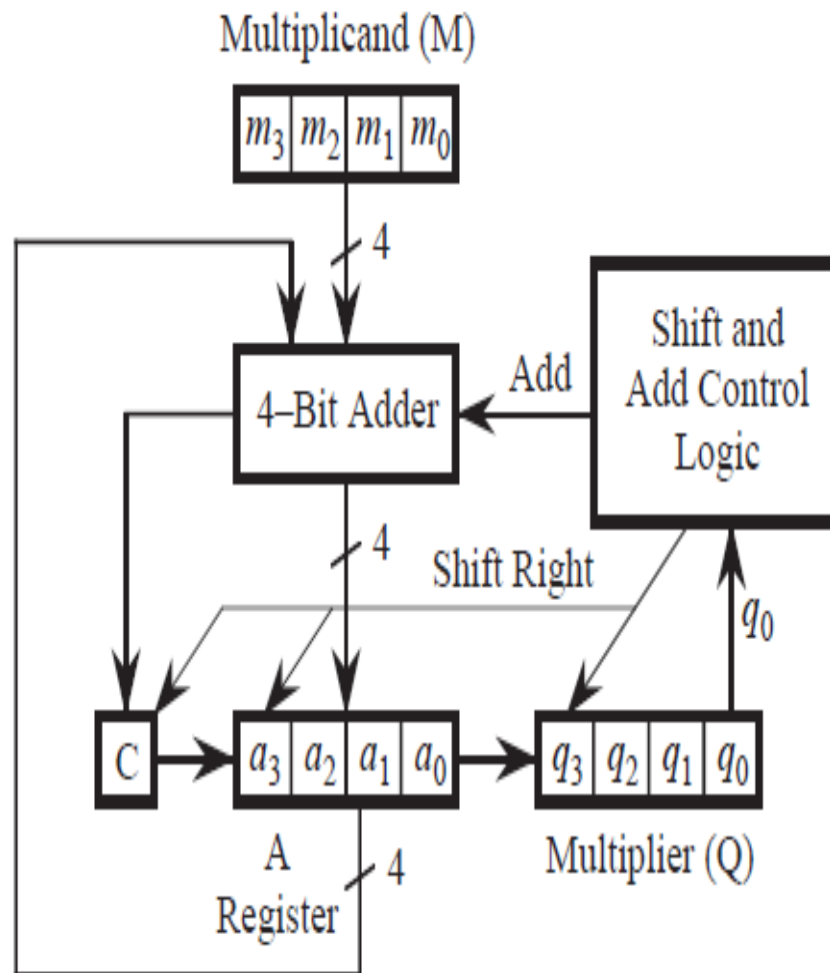
MULTIPLICATION cont...

- The multiplier and multiplicand are loaded into two registers (Q and M).
- A third register, the A register, is also needed and is initially set to 0.
- There is also a 1-bit C register, initialized to 0, which holds a potential carry bit resulting from addition.

Unsigned Binary Multiplication (Right Shift Procedure) using A Serial Multiplier

- The figure in the next slide shows a layout of a multiplication unit for four-bit numbers, in which there is a four-bit adder, a control unit, three four-bit registers, and a one-bit carry register.
- In order to multiply two numbers, the multiplicand is placed in the M register, the multiplier is placed in the Q register, and the A and C registers are cleared to zero.
- During multiplication, the rightmost bit of the multiplier determines whether the multiplicand is added into the product at each step.
- After the multiplicand is added into the product, the multiplier and the A register are simultaneously shifted to the right.

A layout of a multiplication unit for four-bit numbers



Unsigned Binary Multiplication (Right Shift Procedure) using A Serial Multiplier

- The unsigned numbers to be multiplied are placed in registers 'M' and 'Q' respectively
- Register 'A' is initially set to '0'
- Each multiply step has two parts:
 - If the least significant bit (lsb) of 'Q' is 1, then register 'M' is added to 'A': otherwise 00..00 is added to A. The sum is placed back to 'A'
 - Register 'A' and 'Q' are shifted right, with the carry-out of the sum being moved in the high-order bit (msb) of 'A', the low-order bit (lsb) of 'A' being moved into register 'Q', and the rightmost bit of 'Q', which is not used in rest of the algorithm being shifted out.
- After 'n' steps, the product appears in register 'Q' and 'A', with 'A' holding the lower-order bits.

Execution of Multiplication Example

- Multiply $1101_2 \times 1011_2$

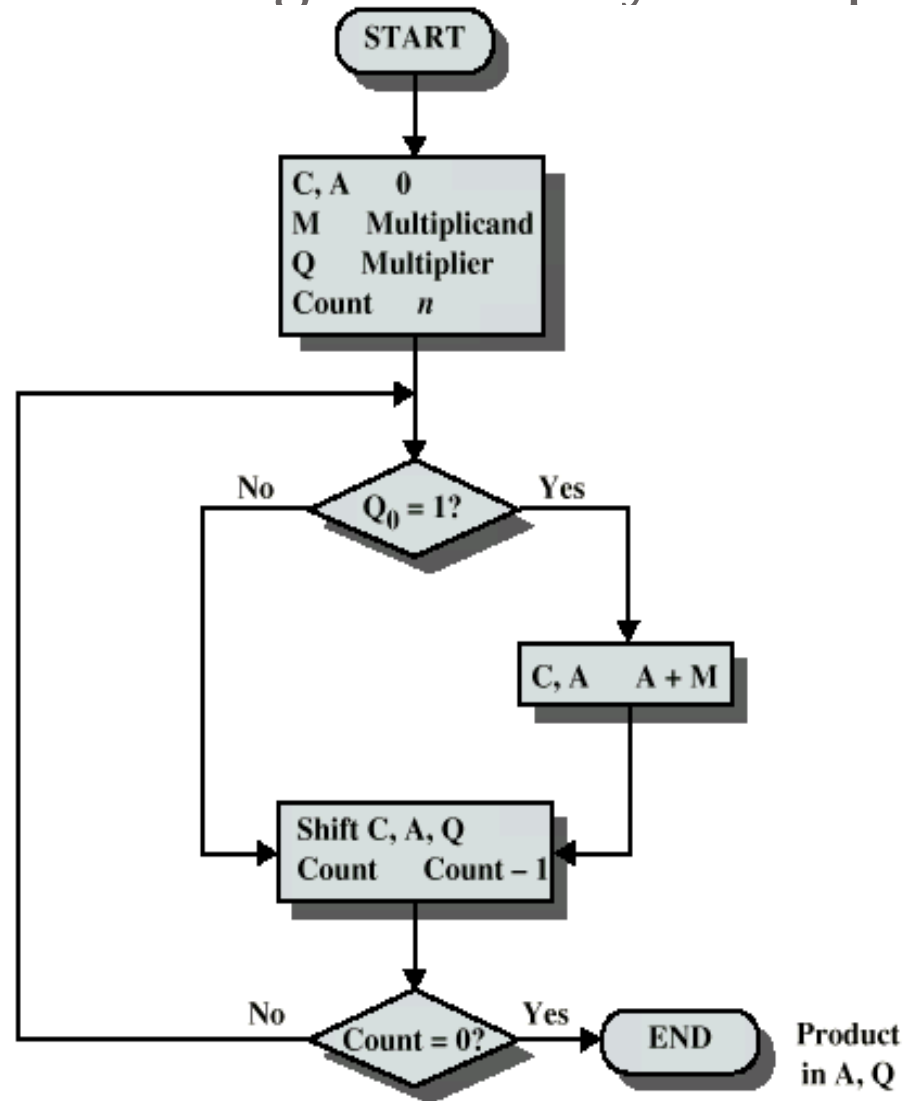
Register M: 1 1 0 1 -----Multiplicand (M), Initial value

		Register	Register	
M	C	'A'	'Q'	
1101	0	0 0 0 0	1 0 1 1	----- Initial values. Note lsb of Q is 1, then
	0	1 1 0 1	1 0 1 1	----- 1(a): add M to A, and sum to A
	0	0 1 1 0	1 1 0 1	-----1(b): Shift one step right. Note Lsb of Q is 1, then
	1	0 0 1 1	1 1 0 1	-----2(a): Add M to A, and sum to A
	0	1 0 0 1	1 1 1 0	-----2(b): Shift one step right, Note lsb is 0, then
	0	1 0 0 1	1 1 1 0	-----3(a): Add 0000 to A, and sum to A, or skip addition
	0	0 1 0 0	1 1 1 1	-----3(b): Shift one step right, lsb of Q is 1, then
	1	0 0 0 1	1 1 1 1	-----4(a): Add M to A, and sum to A
	0	1 0 0 0	1 1 1 1	-----4(b): Shift one step right

- After n- steps, the product appear in register 'A' and 'Q'

Answer = $10001111_2 = 143_{10}$

Flowchart for Unsigned Binary Multiplication

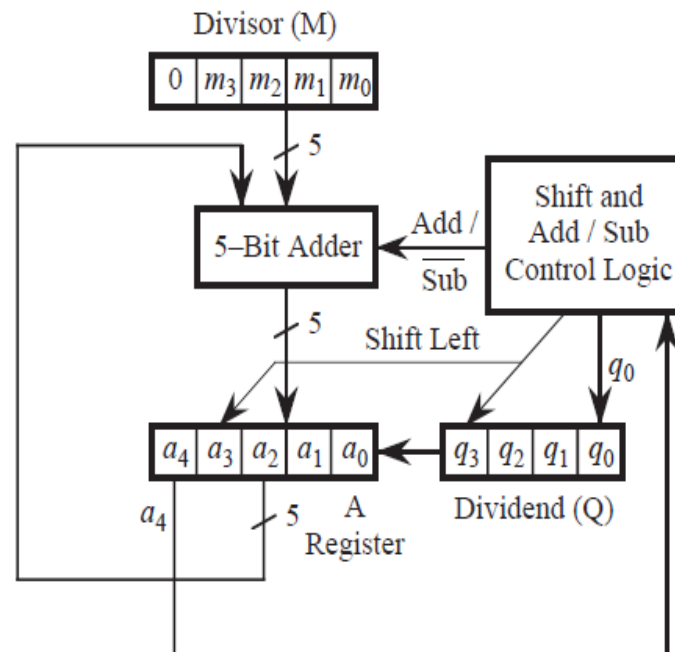


UNSIGNED DIVISION

- In the division algorithm, instead of shifting the product to the right as we did for multiplication, we now shift the quotient to the left, and we subtract instead of adding.
- When two n -bit unsigned numbers are being divided, the result is no larger than n bits.
- The figure below shows a layout of a division unit for four-bit numbers in which there is a five-bit adder, a control unit, a four-bit register for the dividend Q , and two five-bit registers for the divisor M and the remainder A .

UNSIGNED DIVISION cont...

- Five-bit registers are used for A and M, instead of 4-bit registers as we might expect, because an extra bit is needed to indicate the sign of the intermediate result.



Serial Divider

- From $a \div b$ (a/b), put 'a' in register Q, 'b' in register M, 0 in register A and then perform 'n' divide steps
- Each divide step consists of four parts:
 - Shift the register pair (A,Q) one bit left
 - Subtract the content of register M (which has number b) from register A, putting the result back into A
 - If the result of step 2 is negative, set the low-order bit of Q to 0, otherwise to 1
 - If the result of step 2 is negative, restore the old value of A by adding the contents of register M back into A
- Repeat this process n times
- The Q register will contain the quotient, and the A register will contain the remainder

Execution of Division Example

- Divide $14/3 \Rightarrow 1110_2/0011_2$

Register M: 0 0 0 1 1 -----Divisor (M), Initial value

Register 'A'	Register 'Q'
0 0 0 0 0	1 1 1 0 -----Initial values.
0 0 0 0 1	1 1 0 -----1(i): shift one step left
1 1 1 1 0	1 1 0 -----1(ii): subtract A-M, result put in A.
1 1 1 1 0	1 1 0 0 -----1(iii): msb = 1, result is -ve, set Quotient bit Q_0 to 0
0 0 0 0 1	1 1 0 0 -----1(iv): step 2 is -ve, restore the old value of A
0 0 0 1 1	1 0 0 -----2(i): shift left
0 0 0 0 0	1 0 0 -----2(ii): subtract A-M, result in A
0 0 0 0 0	1 0 0 1 -----2(iii): msb = 0, +ve result, set Q_0 to 1
0 0 0 0 1	0 0 1 -----3(i): shift left
1 1 1 1 0	0 0 1 -----3(ii): subtract A-M result in A
1 1 1 1 0	0 0 1 0 -----3(iii): msb = 1, -ve result, Q_0 (lsb) = 0
0 0 0 0 1	0 0 1 0 -----3(iv): Step 2 -ve, restore the old value of A
0 0 0 1 0	0 1 0 -----4(i): Shift left
1 1 1 1 1	0 1 0 -----4(ii): subtract A-M, result in A
1 1 1 1 1	0 1 0 0 -----4(iii): msb = 1, -ve result, Q_0 (lsb) = 0
0 0 0 1 0	0 1 0 0 -----4(iv): restore the old value of A

- After n- steps, the Quotient appear in register 'Q' and Remainder in register 'A'
 $= 0100_2 \text{ rem } 00010_2 = 4_{10} \text{ rem } 2_{10}$

Answer

Flowchart for Unsigned Binary Division

